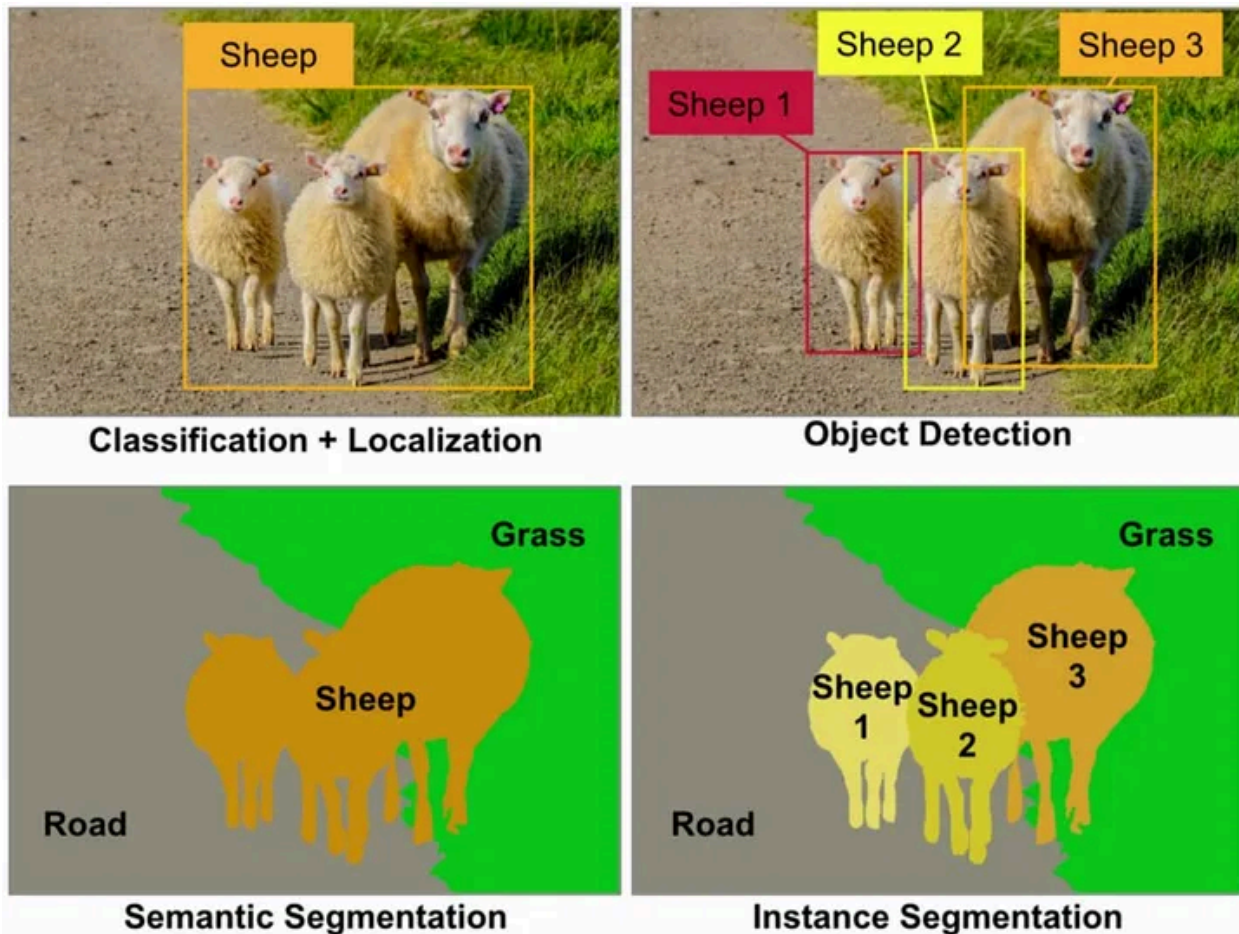


Segmentation

Object detection finds bounding boxes around objects and classifies them. Instance segmentation adds, for every detected object, a pixel mask that gives the shape of the object. Semantic segmentation, on the other hand, does not detect specific instances of objects but classifies every pixel of the image into a category like "road," "sky," or "people."

Practical Machine Learning for Computer Vision, pg 156



Both types of segmentation **create masks**, but instance segmentation goes one step further into finding regions within an image where each identified object is treated as a unique instance. This means it not only distinguishes between different types of objects (like semantic segmentation), but also differentiates multiple instances of the same type within the image, allowing for more precise and individualized analysis.

YOLOV8N-seg

The **YOLOv8n-Seg** model is a variant of the YOLOv8 model specialized for instance segmentation tasks. This model builds upon the foundations of the YOLO object detection architecture, enhancing its capabilities to not only detect objects within an image but also to generate pixel-wise masks for each detected object, effectively

separating them from the background.

Mask Generation Steps:

1. **Image Input:** The image is fed into the model.
2. **Object Detection:** YOLOv8n-Seg first identifies and locates objects within the image.
3. **Segmentation Branch:** In parallel to detection, a separate branch within the model predicts a segmentation mask for each detected object.
4. **Mask Decoding:** The predicted masks are decoded to generate pixel-wise labels, separating the object from the background.

SAM

(SAM was created by META, its license is **Apache License 2.0**)

<https://segment-anything.com/demo>

Unlike YOLOv8 which focuses on object detection, SAM specializes in image segmentation. Its objective is to create precise masks around objects in an image, outlining their exact shapes. SAM can also handle various segmentation tasks through the use of prompts. These prompts can be in the form of text descriptions, clicks on the image, or bounding boxes. This flexibility allows users to segment objects in diverse ways without needing extensive retraining.

Processing Steps:

1. **Image Input:** The process starts with the input image being fed into the model.
2. **Image Encoding:** An image encoder within SAM transforms the image into a series of vectors, essentially capturing the image's visual features.
3. **Prompt Processing:** The user-provided prompt, which can be text descriptions, clicks on the image, or bounding boxes, is also processed by a dedicated section of the model. This translates the prompt into a format the model can understand.
4. **Fusion and Decoding:** The encoded image features and the processed prompt information are then combined (fused) within the model. This combined

representation is used by a decoder to predict the final segmentation mask.

FAST SAM

(Fast SAM itself is **Apache License 2.0**, however because it uses YOLOv8, its license becomes APGL)

FastSAM is designed as a faster alternative to the powerful SAM (Segment Anything Model). It tackles the same task of image segmentation, creating masks around objects in an image. To achieve this speed boost, FastSAM utilizes a simpler architecture based on Convolutional Neural Networks (CNNs), specifically YOLOv8. This results in some trade-off in terms of segmentation accuracy - the masks might not be as precise as those generated by SAM. While SAM excels in high-fidelity segmentation, FastSAM prioritizes speed, making it ideal for tasks where real-time processing is essential.

- **Image Input:** Similar to SAM, the image is fed into the model.
- **Feature Extraction:** YOLOv8 extracts features from the image, identifying patterns and edges that help distinguish objects.
- **Segmentation Branch:** A dedicated branch within YOLOv8 predicts segmentation masks. This branch might use techniques like k-means clustering to group pixels belonging to the same object.
- **Prompt Integration (Optional):** While core functionality relies less on prompts, some variations of FastSAM might incorporate basic prompt information to refine the segmentation based on user intent (e.g., focus on a specific object within the detected ones).

Okay that's cool, but what's the difference between detection and segmentation?

Purpose

- **Detection** focuses on identifying objects within an image and bounding them with rectangles (bounding boxes). The primary goal is to locate objects and classify them.
- **Segmentation**, on the other hand, aims to identify not just where objects are, but also to precisely outline their shapes. It assigns a label to each pixel in the image,

making it clear which object each pixel belongs to.

Details

- **Detection** provides a coarser understanding of the scene since it only locates objects without providing details about their exact boundaries.
- **Segmentation** provides a much finer level of detail by mapping the precise contours of objects, which is particularly useful in applications requiring high accuracy in shape and location, like medical imaging or autonomous driving.

Applications

- **Detection** is typically used in applications where knowing the presence and rough location of objects is sufficient, such as surveillance, simple counting tasks, or when objects are not closely interacting.
- **Segmentation** is used in scenarios where precise spatial understanding is crucial, such as in robot navigation, complex scene understanding, and detailed image editing.

Resources

- **Detection** generally requires less computational power than segmentation because it deals with less granular data (bounding boxes instead of pixel-wise classifications).
- **Segmentation** requires more computational resources due to its detailed nature, processing every pixel in the image for accurate object boundaries.

Comparison

Model	Size	Parameters	Speed (CPU)
Meta's SAM-b	358 MB	94.7 M	51096 ms/im
MobileSAM	40.7 MB	10.1 M	46122 ms/im
FastSAM-s with YOLOv8 backbone	23.7 MB	11.8 M	115 ms/im
Ultralytics YOLOv8n-seg	6.7 MB (53.4x smaller)	3.4 M (27.9x less)	59 ms/im (866x faster)

Documentation

<https://docs.ultralytics.com/models/sam/>

<https://docs.ultralytics.com/tasks/segment/>

Reading

<https://towardsdatascience.com/see-what-you-sam-4eea9ad9a5de>

<https://www.analyticsvidhya.com/blog/2024/07/metasegment-anything-model/>

Application

YOLOV8 + SAM

https://colab.research.google.com/drive/1k9wyFE_wHk_ojreXbK_bkvf0ZZ1nM_AV

Good examples (Segmentation)

<https://www.kaggle.com/code/farzadnekouei/pothole-segmentation-for-road-damage-assessment>